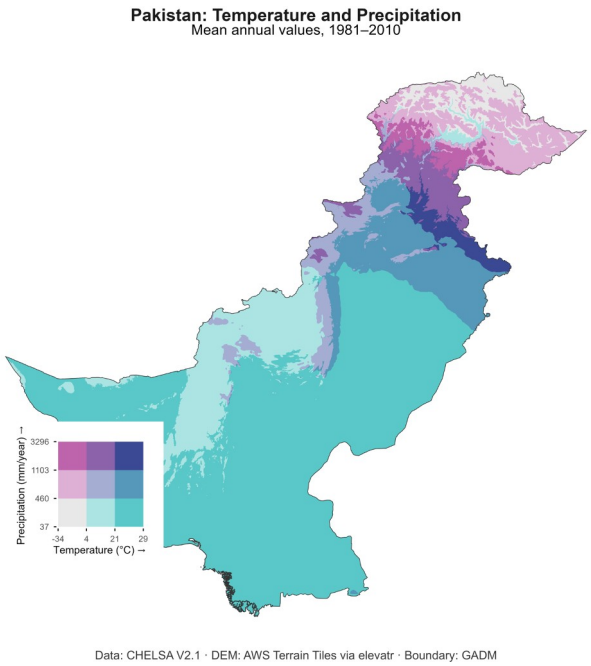
Bivariate Climate Mapping in R

Temperature × precipitation, from installing R to a 3D map

A hands-on course guide for students who know Google Earth Engine and are new to R

Worked example: Pakistan · Practice regions: India, Saudi Arabia, China, Central Asia



The map you will build in this session.

Contents

1. About this guide 3
- What you will be able to do 3
- What you get 3
- Suggested session plan 3

2. The core idea: what a bivariate map is 4
- The pipeline 4
- R vs Google Earth Engine at a glance 5

3. Stage 0 — Install R and RStudio 6
- Install 6
- Find your way around RStudio 6
- Open the project 6
- Five pieces of R syntax you need today 6

4. Stage 1 — Settings and packages 7
- The Settings block 7
- Installing and loading packages 7

5. Stage 2 — Country boundaries 8

6. Stage 3 — Climate data from CHELSA 9

7. Stage 4 — Integrate: stack, crop, mask and check 10

8. Stage 5 — Digital elevation model 11
- Choosing the zoom level 11

9. Stage 6 — Resample and reproject 12
- Resample: same projection, new grid 12
- Reproject: new coordinate reference system 12

10. Stage 7 — From raster to table, and bivariate classes 13
- Raster to table 13
- Bivariate classification 13
- Legend break values 13

11. Stage 8 — The 2D bivariate map 14
- A custom theme 14
- Build the map in layers 14
- Legend and final layout 15

12. Stage 9 — The 3D bivariate map 16
- Prepare the height layer 16
- Build the scene and render 16

13. Results: what the Pakistan map shows 17

14. Practice: other regions 19
- Questions to answer for your region 21
- Instructor key: what students should find 21
- Make your own region 22

15. Troubleshooting 23

16. Glossary and data sources 24
- Data sources 24

1. About this guide

This guide takes you from a computer with no R installed to two publication-quality maps of mean annual temperature and precipitation: a flat 2D bivariate map and a 3D shaded-relief version. Every stage explains what the code does, why it is needed, and how it maps onto Google Earth Engine (GEE) ideas you already know.

What you will be able to do

- Install R, RStudio and R packages, and organise a reproducible project folder.
- Download country boundaries, global climate rasters and elevation data from R.
- Stack, crop, mask, resample and reproject rasters with the terra package.
- Classify two variables into a 3×3 bivariate scheme with Fisher natural breaks.
- Build a 2D bivariate map with ggplot2, biscale and cowplot.
- Turn the map into a 3D ray-traced scene with rayshader and HDRI lighting.
- Adapt the workflow to any country or group of countries by changing six settings.

What you get

File	What it is
`bivariate_map_pakistan.R`	Worked example used in this guide.
`bivariate_map_india.R`, … `_saudi_arabia.R`, …`_china.R`, … `_central_asia.R`	Practice scripts. Identical code; only the Settings block differs.
`teaching/code_walkthrough.txt`	Line-by-line explanation of every function in the script.
`bivariate_maps.Rproj`	RStudio project file. Open this first.
`data/`, `outputs/`	Downloads are cached in data/; maps are saved to outputs/2d and outputs/3d.

Suggested session plan

Part	Stages	Time
1. Setup	Install R and RStudio, open the project, install packages	20 min
2. Getting data	Boundaries, CHELSA climate, DEM	30 min
3. Processing	Stack, crop, mask, resample, reproject, raster → table	30 min
4. 2D bivariate map	Classification, theme, legend, export	30 min
5. 3D map	rayshader scene, HDRI lighting, render	30 min
6. Practice	Each student runs one regional script and interprets it	30 min

> **Tip for the instructor**
>
> Run each script once before class. The global CHELSA files (~770 MB) and the DEMs are then cached in `data/`, and students who copy the whole folder skip the downloads.

2. The core idea: what a bivariate map is

A normal map shows **one** variable with one colour ramp. A bivariate map shows **two** variables at once:

1. Split temperature into 3 classes: low, medium, high.
2. Split precipitation into 3 classes: low, medium, high.
3. Every pixel now falls into one of 3 × 3 = 9 combinations, written as `temperature class-precipitation class`, e.g. `3-1` = hot and dry, `1-3` = cold and wet.
4. Each combination gets its own colour from a 3×3 palette, and a small square legend explains the colours.

This answers a question neither single map can: **where do particular climate combinations occur?** The 3D version keeps the same colours but raises the surface with an elevation model, which shows **why** the pattern looks the way it does: in mountainous countries, elevation controls both temperature and precipitation.

	Low precipitation (1)	Medium (2)	High precipitation (3)
Low temperature (1)	1-1 cold, dry	1-2	1-3 cold, wet
Medium (2)	2-1	2-2 moderate	2-3
High temperature (3)	3-1 hot, dry	3-2	3-3 hot, wet

The pipeline

1. Download global temperature and precipitation rasters (CHELSA) and the country boundary (GADM).
2. Stack the two rasters, then crop and mask them to the country.
3. Download a digital elevation model (DEM) and resample the climate data onto its grid.
4. Reproject everything to an equal-area projection.
5. Convert the raster to a table and classify each pixel into one of 9 bivariate classes.
6. Draw the 2D map with a legend; then use the same classes for colour and the DEM for height to render a 3D map.

R vs Google Earth Engine at a glance

Idea	Google Earth Engine	R (this course)
Raster image	`ee.Image`	`terra::rast()` → SpatRaster
Vector features	`ee.FeatureCollection`	`sf` object / terra SpatVector
Country boundary	FAO GAUL / LSIB collections	`geodata::gadm()`
Add bands	`image.addBands()`	`c(raster1, raster2)`
Clip	`image.clip(geometry)`	`terra::crop(..., mask = TRUE)`
Resample	`image.resample('bilinear')`	`terra::resample(..., method = 'bilinear')`
Reproject	`image.reproject(crs)`	`terra::project(..., crs)`
Statistics	`reduceRegion(ee.Reducer.minMax())`	`terra::global(..., range)`
Display	`Map.addLayer()`	`terra::plot()` / ggplot2
Export	`Export.image.toDrive()`	`ggsave()` / `writeRaster()`
Where it runs	Google's servers	Your own computer

3. Stage 0 — Install R and RStudio

R is the programming language. **RStudio** is the editor you write and run R code in, like the GEE Code Editor.

Install

1. Download R for Windows from **cran.r-project.org** (Download R for Windows → base) and run the installer with the default options.
2. Download **RStudio Desktop** from **posit.co/download/rstudio-desktop** and install it.
3. Optional, Windows command line: `winget install RProject.R` and `winget install Posit.RStudio` do the same.

Find your way around RStudio

Pane	Use
Source (top left)	Your script. Run the current line or selection with **Ctrl + Enter**.
Console (bottom left)	Where code runs and messages appear. You can type commands here too.
Environment (top right)	Every object you have created (rasters, tables, plots).
Files / Plots / Help (bottom right)	Browse the folder, see plots, read help. Type `?terra::crop` in the console for help on a function.

Open the project

Copy the whole `bivariate_maps` folder to your computer, then double-click `bivariate_maps.Rproj`. RStudio opens with the project folder as the working directory, so paths like `data/chelsa` work without typing full paths. Then open `bivariate_map_pakistan.R` from the Files pane.

Five pieces of R syntax you need today

Syntax	Meaning
`x <- 5`	Store a value in a variable (like `var x = 5` in GEE).
`c(1, 12)`	Combine values into a vector.
`terra::crop()`	Call `crop()` from the terra package.
`a |> f(b)`	The pipe: same as `f(a, b)`. Like method chaining in GEE.
`# text`	A comment; R ignores it.

Try it

In the console, type `x <- c(10, 20, 30)` and then `mean(x)`. Then type `x |> mean()`. Both give 20.

4. Stage 1 — Settings and packages

The Settings block

Every script starts with a Settings block. This is the **only** part that changes between regions:

```
region_name <- "Pakistan"     # used in the map title
region_slug <- "pakistan"     # used in file names
country_iso <- "PAK"          # one or more ISO3 codes

# Lambert Azimuthal Equal Area centred on the region
crs_laea <- "+proj=laea +lat_0=30 +lon_0=70 +datum=WGS84 +units=m +no_defs"

dem_zoom   <- 7    # elevation detail; lower for bigger regions
dem_floor  <- 0    # lowest believable elevation (m)
fig_width  <- 7    # output size in inches
fig_height <- 7
```

Installing and loading packages

Packages are add-ons that give R new functions. You install a package once, and load it in every session. `pacman::p_load()` does both: it installs anything missing, then loads everything.

```
if (!requireNamespace("pacman", quietly = TRUE)) {
    install.packages("pacman", repos = "https://cloud.r-project.org")
}

pacman::p_load(
    geodata, tidyverse, sf, terra, biscale, elevatr,
    cowplot, gridGraphics, rayshader, magick
)
pacman::p_load_gh("inSileco/rchelsa")   # a package hosted on GitHub
```

Package	Role
terra	Raster processing: read, crop, mask, resample, reproject
sf	Vector data (polygons) as tables
geodata	Country boundaries from GADM
elevatr	Elevation (DEM) tiles for any area
rchelsa	CHELSA climate downloads (used as a fallback)
tidyverse	Data handling and ggplot2 for maps
biscale	Bivariate classes, palettes and legends
cowplot, gridGraphics	Combine the map and legend into one figure
rayshader	3D maps with ray-traced shadows
magick	Image editing: add legend and title to the 3D render

Check before moving on

The first install takes 5–10 minutes. Afterwards, `p_load()` finishes in seconds with no errors.

5. Stage 2 — Country boundaries

GADM is a global database of administrative boundaries. Level 0 is the national outline, level 1 provinces, level 2 districts.

```
country_list <- lapply(country_iso, function(iso) {
    geodata::gadm(country = iso, level = 0, path = dir_boundaries)
})
country_vect <- terra::aggregate(do.call(rbind, country_list))
country_sf   <- sf::st_as_sf(country_vect) |>
    sf::st_cast("MULTIPOLYGON")
```

- `lapply()` downloads one boundary per ISO code, so the same code handles one country or five.
- `do.call(rbind, …)` stacks them; `terra::aggregate()` dissolves everything into one outline.
- GADM returns Pakistan as **two** features: PAK plus Z06 (Pakistan-administered Kashmir). India and China also come with extra "Z" pieces. Without dissolving them, later steps fail.
- `st_as_sf()` converts to the sf format that ggplot2 and elevatr expect; `st_cast("MULTIPOLYGON")` makes the geometry type explicit.

Coming from Google Earth Engine

```
ee.FeatureCollection("FAO/GAUL/2015/level0").filter(ee.Filter.eq("ADM0_NAME",
"Pakistan")).union()
```

Try it

Run `plot(country_vect)` in the console. Then change `level = 0` to `level = 1` and plot again to see the provinces.

Note on boundaries

GADM shows disputed areas in its own way; any boundary dataset is a political choice. Mention the source on every map.

6. Stage 3 — Climate data from CHELSA

CHELSA provides global climate grids at 30 arc-seconds (~1 km). We use two of its 19 bioclimatic variables, averaged over **1981–2010** (version 2.1):

ID	Variable	Unit
bio1	Annual mean air temperature	°C
bio12	Annual precipitation amount	mm per year

```
chelsa_ids <- c(1, 12)
chelsa_v2_url   <- "https://os.zhdk.cloud.switch.ch/chelsav2/GLOBAL/climatologies/1981-2010/bio"
chelsa_v2_files <- paste0("CHELSA_bio", chelsa_ids, "_1981-2010_V.2.1.tif")

for (f in chelsa_v2_files) {
    destfile <- file.path(dir_chelsa, f)
    if (!file.exists(destfile)) {
        download.file(file.path(chelsa_v2_url, f), destfile = destfile, mode = "wb")
    }
}

temp <- terra::rast(file.path(dir_chelsa, chelsa_v2_files[1]))
prec <- terra::rast(file.path(dir_chelsa, chelsa_v2_files[2]))
```

- The files are **global** (115 MB and 655 MB). They are downloaded once and reused by every region script.
- `mode = "wb"` (write binary) is required on Windows, or the GeoTIFF is corrupted.
- `terra::rast()` opens the file lazily: R reads only the pixels it needs, so even a global file opens instantly.
- The full script wraps the download in `tryCatch()` and falls back to `rchelsa::get_chelsea_data()` if CHELSA's V2.1 server is down. That function serves the older V1.2 (1979–2013, temperature stored as °C × 10), and the script corrects for it.

Why we don't divide precipitation by 30

bio12 is already the **average yearly total** over 1981–2010. Dividing by 30 would not produce an annual average; it would only shrink the numbers. The Fisher classes, and therefore the map colours, would not even change. We keep real mm/year so the legend is correct.

Coming from Google Earth Engine

`ee.Image("...CHELSA...")`. In GEE the data already lives on Google's servers; in R you download it once and process it locally.

7. Stage 4 — Integrate: stack, crop, mask and check

```
temp_prec <- c(temp, prec)
names(temp_prec) <- c("temp", "prec")

temp_prec_country <- terra::crop(temp_prec, country_vect, snap = "in", mask = TRUE)

terra::global(temp_prec_country, range, na.rm = TRUE)
```

- `c(temp, prec)` stacks two rasters into one raster with two layers (bands).
- `crop` cuts the raster to the country's bounding box; `mask = TRUE` sets every pixel outside the border to NA.
- We crop **first**, while the data is still in its original lon/lat grid, so every later step works on a small piece instead of the whole globe.
- `terra::global(…, range)` prints the minimum and maximum of each layer: a quick sanity check.

Coming from Google Earth Engine

`temp.addBands(prec).clip(country)` followed by `reduceRegion(ee.Reducer.minMax())`.

Check before moving on

For Pakistan: temperature **–35 to 29 °C**, precipitation **37 to 3,316 mm/year**.

Temperatures in the hundreds would mean the data is in different units (e.g. °C × 10 or Kelvin × 10). The script stops automatically if values look wrong.

8. Stage 5 — Digital elevation model

```
dem_raw <- elevatr::get_elev_raster(
    locations = country_sf,
    z = dem_zoom,
    clip = "locations"
)
dem <- terra::rast(dem_raw)
dem <- terra::clamp(dem, lower = dem_floor, values = TRUE)
```

- elevatr downloads AWS Terrain Tiles (a blend of SRTM, GMTED and other sources) and clips them to the country.
- The script saves the DEM to `data/dem/` and skips the download next time.
- `clamp()` removes impossible values. The Pakistan DEM contained **−1,687 m** at one edge, a tile artefact; Pakistan's lowest point is sea level.

Choosing the zoom level

Like web-map zoom: each step up halves the cell size and multiplies the number of cells by about 4. Big regions need a lower zoom, or the download and the 3D render become too large.

Region	Area	dem_zoom	Cell size	dem_floor
Pakistan	~0.9M km²	7	~500 m	0 m
India	~3.3M km²	6	~1 km	−5 m
Saudi Arabia	~2.1M km²	6	~1 km	0 m
China	~9.6M km²	5	~2 km	−160 m (Turpan)
Central Asia (5 countries)	~4M km²	5	~2 km	−140 m (Karagiye)

Coming from Google Earth Engine

`ee.Image("USGS/SRTMGL1_003").clip(country)`. The zoom level plays the role of the `scale` parameter.

9. Stage 6 — Resample and reproject

```
temp_prec_resampled <- terra::resample(temp_prec_country, dem, method = "bilinear") |>
    terra::project(crs_laea, method = "bilinear")

terra::plot(temp_prec_resampled)
```

Resample: same projection, new grid

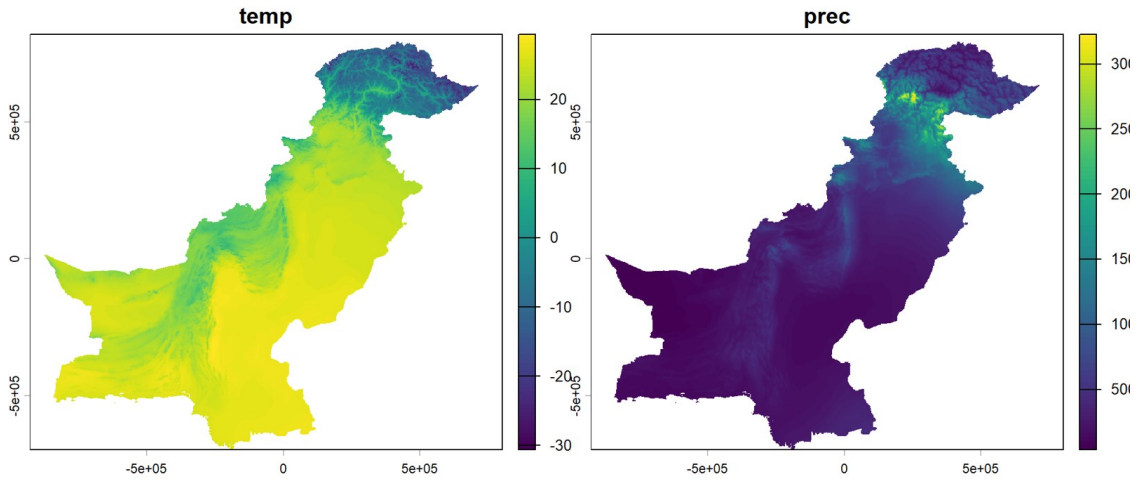
`resample()` moves the climate values onto the DEM's grid. **Bilinear** interpolation gives each new cell a distance-weighted average of the 4 nearest original cells, so values change smoothly instead of in blocks. For Pakistan the DEM (~500 m) is finer than CHELSA (~1 km), so the map looks smoother. More importantly, climate and elevation now share one grid, which the 3D step needs.

Reproject: new coordinate reference system

Latitude/longitude degrees distort shapes and areas. **Lambert Azimuthal Equal Area (LAEA)** keeps areas true, which matters when you compare how much land falls in each climate class. Setting `lat_0` and `lon_0` to the region's centre keeps distortion as low as possible.

Coming from Google Earth Engine

```
image.resample("bilinear").reproject({crs: ..., scale: ...})
```



Stage 6 check: the resampled and reprojected layers. Pakistan only, nothing outside the border.

Try it

Change `crs_laea` to `"EPSG:4326"` (plain lon/lat), rerun Stages 6–9 and compare the country's shape.

10. Stage 7 — From raster to table, and bivariate classes

Raster to table

```
temp_prec_df <- as.data.frame(temp_prec_resampled, xy = TRUE, na.rm = TRUE)
head(temp_prec_df)
```

ggplot2 and biscale work with tables, not rasters. Each pixel becomes a row with columns `x`, `y`, `temp`, `prec`; `na.rm = TRUE` drops the empty pixels outside the country. For Pakistan this is about 4 million rows.

Bivariate classification

```
set.seed(2026)   # same random sample, same breaks, on every run

breaks <- biscale::bi_class(
    temp_prec_df,
    x = temp, y = prec,
    style = "fisher", dim = 3
)
```

- `bi_class()` adds a column `bi_class` with values `1-1` to `3-3`: first the temperature class, then the precipitation class.
- **Fisher-Jenks natural breaks** place the class limits so each class is as internally similar as possible (minimum within-class variance). This suits skewed data like precipitation, where equal intervals would put almost every pixel in the lowest class.
- On large tables, Fisher estimates the breaks from a random sample, so R prints a harmless "sampling imposed" warning.

Legend break values

```
class_x <- as.integer(substr(breaks$bi_class, 1, 1))
class_y <- as.integer(substr(breaks$bi_class, 3, 3))
legend_breaks <- list(
    bi_x = round(c(min(breaks$temp), tapply(breaks$temp, class_x, max)), 0),
    bi_y = round(c(min(breaks$prec), tapply(breaks$prec, class_y, max)), 0)
)
```

For each class we take the largest value that actually landed in it, plus the overall minimum: 4 boundaries per axis. biscale's own `bi_class_breaks(split = TRUE)` mis-reads negative temperatures (it splits "−34-4" on the minus signs), so we calculate them ourselves.

> **Try it**
>
> Run `table(breaks$bi_class)` to see how many pixels fall in each of the 9 classes. Which class is the largest?

11. Stage 8 — The 2D bivariate map

A custom theme

```
theme_for_the_win <- function() {
    theme_minimal() +
        theme(
            axis.title = element_blank(),
            axis.text = element_blank(),
            panel.grid = element_blank(),
            legend.position = "none",
            plot.background = element_rect(fill = "white", colour = NA),
            plot.title = element_text(
                colour = "grey10", hjust = .5, face = "bold", vjust = -1
            ),
            plot.caption = element_text(
                size = 9, colour = "grey20", hjust = .5, vjust = 1
            ),
            plot.margin = unit(c(t = 0, r = 0, b = 0, l = 0), "lines")
        )
}
```

A theme controls everything that isn't data: background, fonts, margins. Zero margins remove the white border; the ggplot legend is switched off because biscale draws a better one.

Build the map in layers

```
map <- ggplot(breaks) +
    geom_raster(aes(x = x, y = y, fill = bi_class)) +
    biscale::bi_scale_fill(pal = "DkBlue", dim = 3) +
    geom_sf(data = country_sf_laea, fill = NA, colour = "grey20", linewidth = .2) +
    coord_sf(crs = crs_laea) +
    labs(title = map_title, subtitle = map_subtitle, caption = map_caption) +
    theme_for_the_win()
```

Layer	What it does
`ggplot(breaks)`	Start a plot from the classified table.
`geom_raster(aes(…, fill = bi_class))`	One square per pixel, coloured by its class. `aes()` maps columns to visual properties.
`bi_scale_fill(pal = "DkBlue")`	Turn the 9 class codes into the 9 DkBlue colours.
`geom_sf(…)`	Country border on top, no fill.
`coord_sf(crs = crs_laea)`	Tell ggplot the map is in the LAEA projection.

Legend and final layout

```
legend <- biscale::bi_legend(
    pal = "DkBlue", dim = 3, breaks = legend_breaks,
    xlab = "Temperature (°C)", ylab = "Precipitation (mm/year)", size = 8
)

full_map <- cowplot::ggdraw() +
    cowplot::draw_plot(map,    x = 0,   y = 0,   width = 1,   height = 1) +
    cowplot::draw_plot(legend, x = .05, y = .13, width = .25, height = .25)

print(full_map)
ggsave("outputs/2d/bivariate_pakistan_2d.png", full_map,
       width = 7, height = 7, dpi = 600, bg = "white")
```

`ggdraw()` creates an empty canvas where (0, 0) is the bottom-left corner and (1, 1) the top-right. The map fills the canvas; the legend sits 5% from the left and 13% from the bottom and takes a quarter of the width and height. 7 × 7 inches at 600 dpi gives a 4,200 × 4,200-pixel image, which is print quality.

Try it

Change `pal = "DkBlue"` to `"GrPink"` or `"DkViolet"` in both `bi_scale_fill()` and `bi_legend()`, and save again.

12. Stage 9 — The 3D bivariate map

rayshader's `plot_gg()` takes **two** ggplots: one decides the **colours** (our bivariate classes), the other decides the **heights** (the DEM).

Prepare the height layer

```
dem_df <- terra::project(dem, temp_prec_resampled, method = "bilinear") |>
    as.data.frame(xy = TRUE, na.rm = TRUE)
names(dem_df)[3] <- "dem"

map_height <- ggplot(dem_df) +
    geom_raster(aes(x = x, y = y, fill = dem)) +
    scale_fill_gradientn(colours = "white") +
    coord_sf(crs = crs_laea) +
    theme_void() + theme(legend.position = "none")
```

Projecting the DEM onto the climate raster itself (instead of onto a CRS) puts both on exactly the same grid, cell for cell. The colour layer, `map_colour`, is the 2D map without title, border or margins, so both ggplots have identical panels and the colours sit exactly on the mountains.

Build the scene and render

Render time

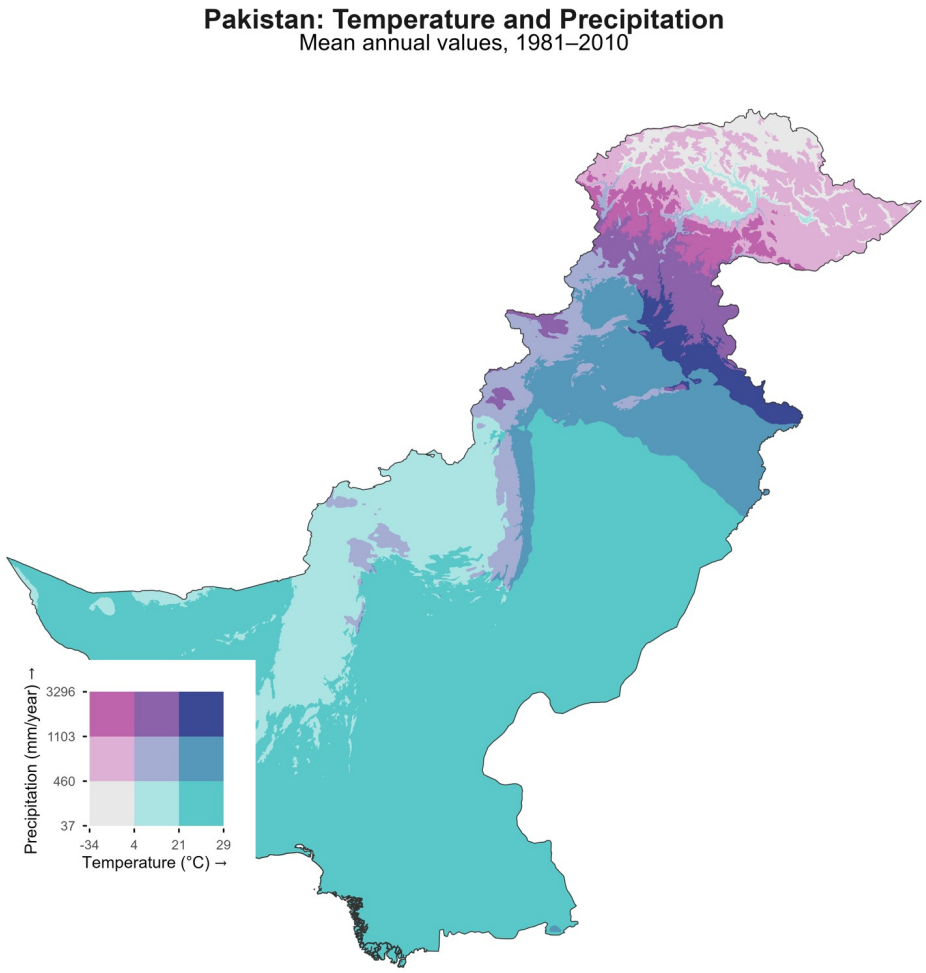
`render_quality <- "test"`: 600 px in 1–3 min, for experimenting. `"final"`: 2,100 px in 10–40 min.

```
rayshader::plot_gg(
    ggobj = map_colour, ggobj_height = map_height,
    width = fig_width, height = fig_height,
    scale = 100, shadow = TRUE, shadow_intensity = 1,
    phi = 87, theta = 0, zoom = .56,
    multicore = TRUE
)

rayshader::render_highquality(
    filename = raw_3d_file,
    light = FALSE,
    environment_light = hdri_file, intensity_env = 1, rotate_env = 90,
    interactive = FALSE, parallel = TRUE,
    samples = render_samples, width = render_width, height = render_height
)
```

Argument	Meaning
`scale`	Vertical exaggeration: bigger = taller relief.
`phi`, `theta`, `zoom`	Camera angle above the horizon (90 = straight down), rotation, and distance.
`multicore = TRUE`	Shadow calculation on all CPU cores (multithreading).
`environment_light`	An HDRI: a 360° photo that stores real light intensities. It lights the model as if it stood in that room. We use Poly Haven's free "photo_studio_loft_hall".
`parallel = TRUE`	Path-tracing on all CPU cores (multiprocessing).
`samples`	Light rays per pixel: more = less grain, but slower. "test" = 64, "final" = 256.
`magick::…`	Afterwards: reads the render back as an image object, pastes the legend (same 5% / 13% position as in 2D), adds title and caption, saves the final PNG.

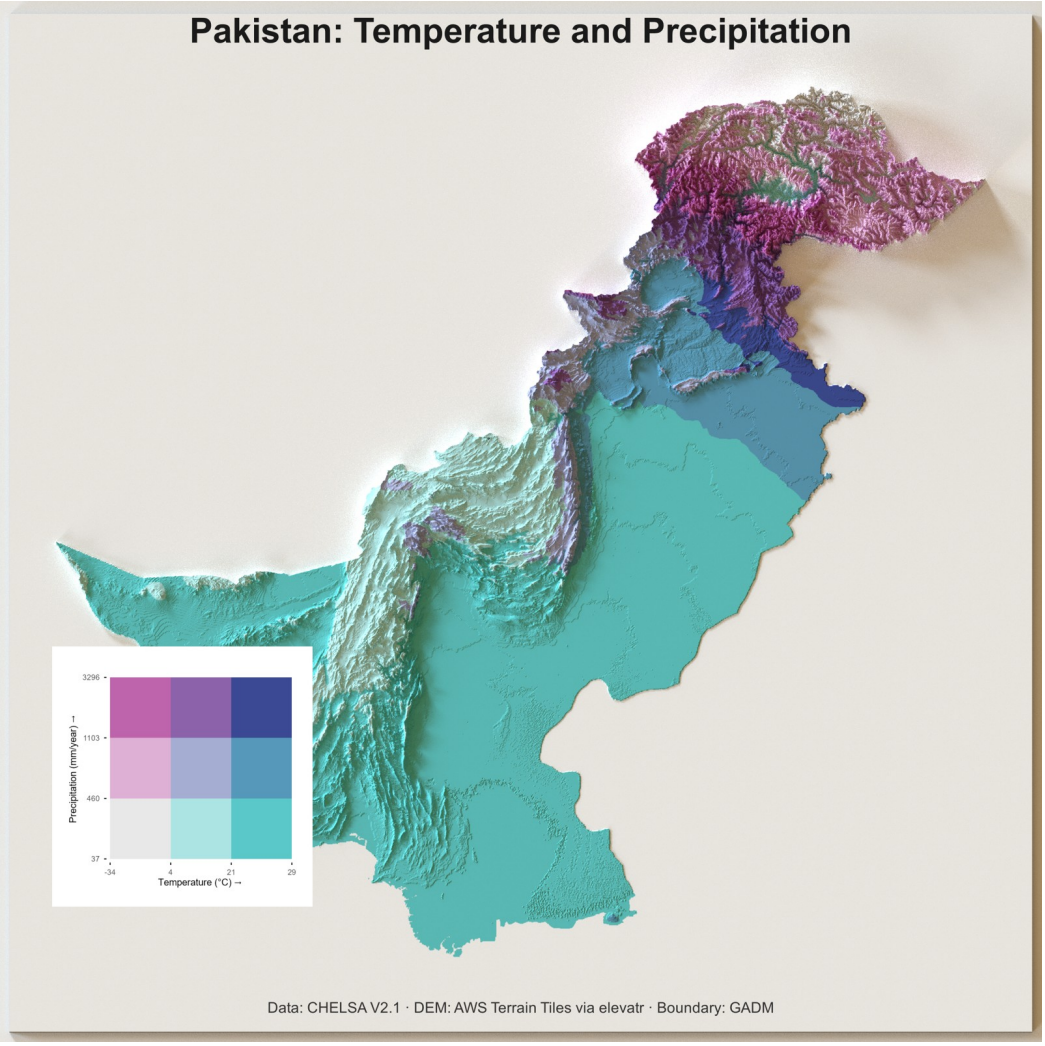
13. Results: what the Pakistan map shows



Data: CHELSA V2.1 · DEM: AWS Terrain Tiles via elevatr · Boundary: GADM

2D bivariate map of Pakistan, mean annual values 1981–2010.

Region	Bivariate class	Explanation
Indus plain, Sindh, southern Balochistan	Hot and dry (3-1, teal)	Low elevation, far from monsoon moisture. Above 21 °C and below 460 mm/year.
Himalayan foothills: Azad Kashmir to the Murree hills	Warm and wet (3-3 / 2-3, dark blue)	Monsoon air is forced up the first mountain ranges it meets (orographic rainfall). Over 1,100 mm/year.
Karakoram, Gilgit-Baltistan	Cold and fairly dry (1-1 / 1-2, grey and pink)	Highest elevations; temperature falls about 6.5 °C per 1,000 m. Below 4 °C.
Sulaiman and Kirthar ranges	Cooler, slightly wetter strips (2-1 / 2-2)	The same orographic effect on a smaller scale, inside the arid west.

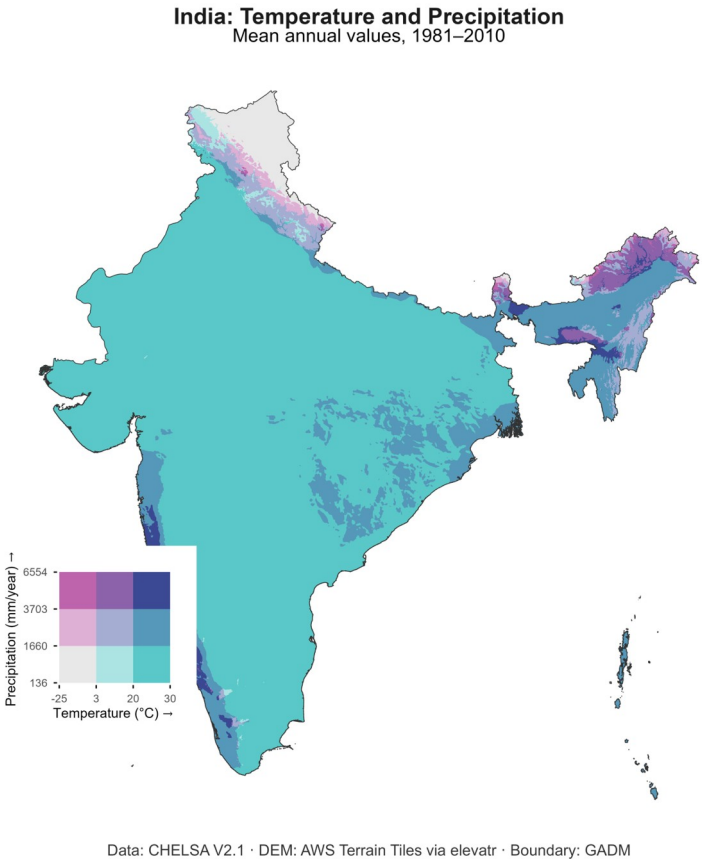


3D version: the colour boundaries follow the ridges and valleys. Elevation controls both variables.

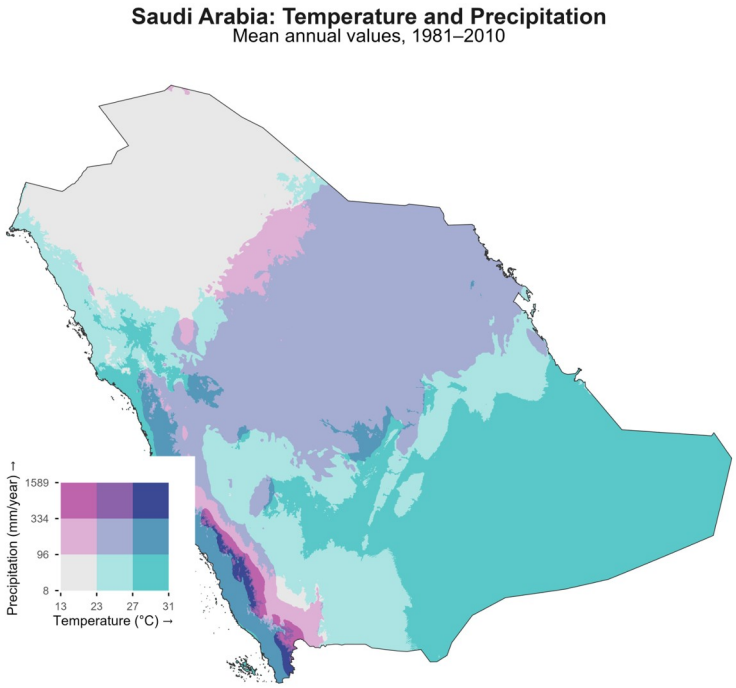
14. Practice: other regions

Each practice script is the Pakistan script with a different Settings block. Open one, run it top to bottom (Ctrl + Shift + Enter, or **Source**), and interpret the result.

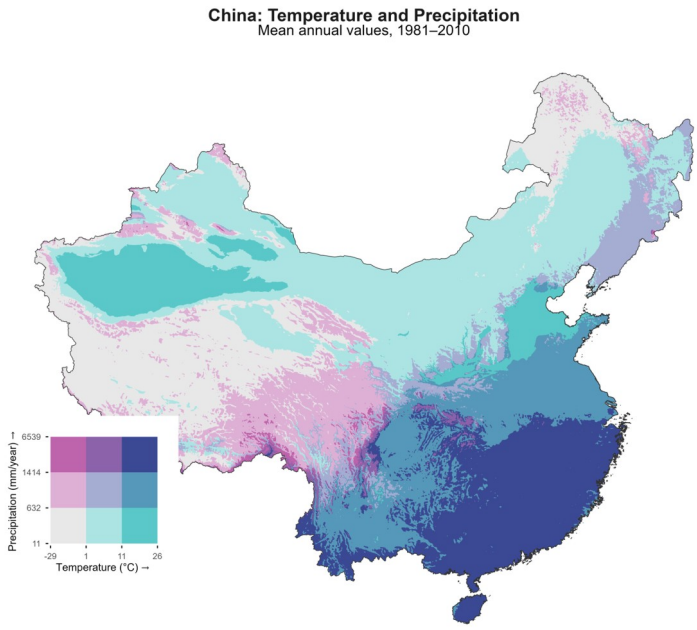
Script	country_iso	Projection centre	dem_zoom	Figure
`bivariate_map_india.R`	`"IND"`	23°N, 82°E	6	7 × 7 in
`bivariate_map_saudi_arabia.R`	`"SAU"`	24°N, 45°E	6	7 × 6 in
`bivariate_map_china.R`	`"CHN"`	36°N, 104°E	5	9 × 7 in
`bivariate_map_central_asia.R`	`c("KAZ", "UZB", "TKM", "KGZ", "TJK")`	45°N, 67°E	5	9 × 6 in



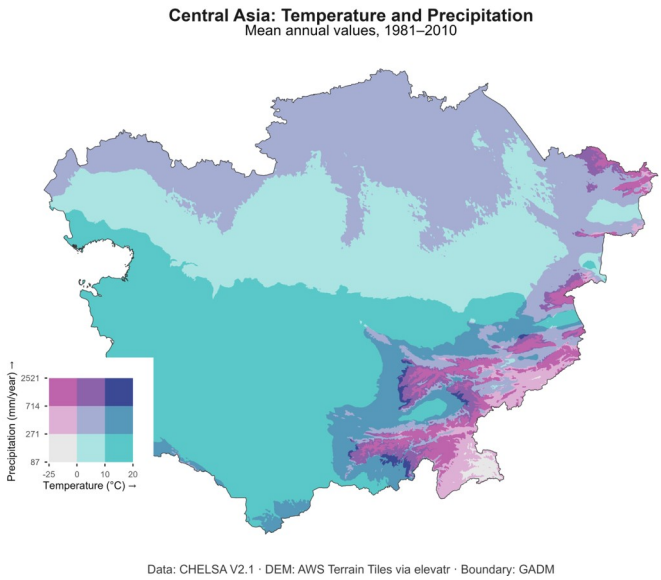
India: expected 2D result.



Saudi Arabia: expected 2D result.



China: expected 2D result.



Central Asia: expected 2D result.

Questions to answer for your region

1. What are the minimum and maximum temperature and precipitation (Stage 4 check)?
2. Which bivariate class covers the largest area? Use `table(breaks$bi_class)`.
3. Where is the wettest class? Which mountain range or coast explains it?
4. In the 3D render, which colour boundaries follow the terrain, and which don't?
5. Note that the legend breaks differ between regions: Fisher breaks adapt to each region's data. Why does that make maps of different regions hard to compare directly?

Instructor key: what students should find

Region	Value range (Stage 4)	Main patterns
India	−27 to 30 °C; 136 to 6,554 mm	Most of the subcontinent is hot with under 1,660 mm. The Western Ghats and Meghalaya / North-East are the wet classes (orographic monsoon rain). Ladakh and the high Himalaya are cold and dry.
Saudi Arabia	12 to 31 °C; 8 to 1,610 mm	Almost all of the country is dry. The Asir mountains in the south-west stand out as the only wet, cooler zone. The Rub' al Khali is hot and dry; the north is cooler and dry.
China	−33 to 27 °C; 11 to 6,554 mm	A sharp south-east / north-west split: the warm, wet south-east (East Asian monsoon) against the cold, dry Tibetan Plateau and the hot, dry Tarim and Junggar basins.
Central Asia	−30 to 20 °C; 87 to 2,598 mm	The hot, dry Turan lowland (Turkmenistan, Uzbekistan) contrasts with the Kazakh steppe. The Tian Shan, Pamir and Altai show as cold and wet: mountains capture the moisture.

A lesson about data types

India and China both report a maximum of exactly **6,553.5 mm**. That is 65,535 × 0.1: CHELSA stores precipitation as 16-bit integers (maximum 65,535) with a scale factor of 0.1, so anything wetter is cut off at that value. The capped cells are in Meghalaya (Cherrapunji, Mawsynram) and the eastern Himalaya, the wettest places on Earth. The classes are unaffected, but the top legend value means "6,554 mm or more". Always ask whether a suspiciously round maximum is real.

Make your own region

1. Copy any script and rename it, e.g. `bivariate_map_iran.R`.
2. Change `country_iso` (look codes up at gadm.org), `region_name` and `region_slug`.
3. Set `lat_0` / `lon_0` in `crs_laea` to the region's centre.
4. Pick `dem_zoom` from the table in Stage 5, and `dem_floor` from the region's lowest point.
5. Set `fig_width` / `fig_height` to match the region's shape.

15. Troubleshooting

Problem	Cause and fix
Download times out or `cannot open URL`	Network problem. Rerun; files that finished downloading are kept.
`not implemented for objects of class sfc_GEOMETRY`	The boundary has mixed geometry types. `aggregate()` + `st_cast("MULTIPOLYGON")` fix it.
`object 'breaks_seq' not found` in `bi_legend()`	The legend breaks have the wrong length. Use the Stage 7 method, not `bi_class_breaks(split = TRUE)`.
Temperatures in the hundreds	Data in °C × 10 (CHELSA V1.2). The script divides by 10 automatically.
3D render has a black background	The colour ggplot needs `plot.background = element_rect(fill = "white")`.
3D render is grainy	Expected in "test" mode. Use `render_quality <- "final"`.
R runs out of memory	Lower `dem_zoom` by 1.
`could not find function`	The package isn't loaded. Rerun the Stage 1 block.

16. Glossary and data sources

Term	Meaning
Raster	A grid of cells, each holding a value (temperature, elevation…).
Vector	Points, lines or polygons, e.g. a country border.
CRS	Coordinate reference system: how coordinates relate to places on Earth.
LAEA	Lambert Azimuthal Equal Area projection; preserves area.
Mask	Setting cells outside a polygon to NA (no data).
Resample	Moving values to a new grid in the same CRS.
Reproject	Transforming data into a different CRS.
Bilinear interpolation	New value = distance-weighted average of the 4 nearest cells.
DEM	Digital elevation model: a raster of ground height.
Fisher-Jenks breaks	Class limits that minimise variance within each class.
HDRI	High-dynamic-range 360° image used as realistic environment lighting.
Ray tracing / path tracing	Simulating light rays to produce shadows and realistic shading.

Data sources

- **CHELSA V2.1**: Karger, D.N. et al. (2017) Climatologies at high resolution for the earth's land surface areas. Scientific Data 4, 170122. chelsa-climate.org
- **GADM 4.1**: Global Administrative Areas, gadm.org
- **AWS Terrain Tiles** via the elevatr package (Hollister et al.)
- **Poly Haven**: free HDRIs, polyhaven.com
- R packages: terra, sf, geodata, elevatr, biscale, ggplot2, cowplot, rayshader, magick